Student	Distributed_Version_Control_Mai_Hung
Started	10/18/2026, 11:53:06 PM
Submitted	10/18/2026, 11:56:39 PM

MODULE 2 — GIT

Distributed Version Control System

WHY THIS MATTERS

THE PROBLEM

Without version control, teams overwrite each other's work, lose history, and cannot roll back broken changes. Emailing zip files or sharing network drives does not scale past a single developer.

THE SOLUTION

A distributed VCS gives every developer a full local copy of the repository. Changes flow through a structured pipeline: edit locally, stage, commit, then push to a shared remote that triggers automation.

REAL-WORLD USE

Git is the de-facto standard used by GitHub, GitLab, and Bitbucket. Every professional software team uses branching strategies (GitFlow, trunk-based) to ship code safely at scale.

YOUR GOAL

Understand the data flow from a local file edit all the way to a remote repository and CI trigger, so you can reason about where conflicts happen and how branches are reconciled.

DESIGN CANVAS — DRAG COMPONENTS IN THE CORRECT ORDER

1. Local Working Tree
2. Staging Index (git add)
3. Local Repository (git commit)
4. Branch Reference
5. Merge / Rebase Engine

6. Remote Repository (git push)

7. CI Trigger (Webhook)

STATUS: PASS / COMPLETED

Distributed VCS architecture verified. Changes flow from the Local Working Tree through the Staging Index and Local Repository, are organized by Branch References, reconciled by the Merge Engine, pushed to the Remote Repository, and finally trigger CI automation.

© 2026 Clark Ngo